

Spring Services, Spring JPA & Spring JDBC

Day 11 focuses on formalizing the three-layer architecture and understanding three different ways to access a relational database.

Topics Covered

- Spring REST API — service-layer design
- Spring JPA — repositories and entity mapping
- Spring JDBC — template-based data access
- Comparison of hand-written JDBC, JdbcTemplate, and Spring Data JPA
- Rebuilding the Employee Payroll App using Spring Data JPA

Expected Outcome

- State clearly what belongs in a Controller, Service, and DAO/Repository.
- Write a Spring Data JPA repository interface with full CRUD and no implementation class.
- Use JdbcTemplate when real SQL control is required without JDBC boilerplate.
- Choose between JDBC, JdbcTemplate, and JPA based on the actual requirement.

1. Service-Layer Design — Formalizing What We've Been Doing

The Three-Layer Responsibility Split

The same pattern has been used in the AppointmentService, ContactService, and EmployeeService. Day 11 turns that pattern into explicit rules.

Responsibility boundaries

Controller

- Reads HTTP input
- Calls Service methods
- Converts results into HTTP responses
- Contains no business logic
- Contains no SQL

Service

- Contains business logic and validation rules
- Coordinates multiple DAO/Repository calls
- Owns transaction boundaries
- Contains no SQL - Contains no HTTP-specific code

DAO / Repository

- Contains SQL or JPA data-access logic
- Knows nothing about business rules or HTTP
- Reads and writes application data

Which Layer Does This Code Belong To?

Code / Action	Correct Layer	Why
salary < 0 → reject	Service	Business rule
@NotBlank on request field	DTO / Controller boundary	Input validation
Transaction boundary decision	Service	Business operation scope
SELECT ... WHERE id = ?	DAO / Repository	Database access
ResponseEntity.status(...)	Controller	HTTP response

Why this matters: each layer should be replaceable without changing the layers above it. A hand-written JDBC DAO can later be replaced with a Spring Data JPA repository without changing the Controller.

✓ Good EmployeeService checks salary rules and then calls the repository.	✗ Bad Controller contains salary business rules and every other caller has to duplicate them.
---	---

Interview Question

Q: Why shouldn't business validation such as "salary must be positive" live in the Controller?

A: The Controller handles HTTP concerns. Business rules must also apply when the same operation is triggered by a scheduled job, message listener, or another internal caller. Keeping the rule in the Service makes the behavior consistent.

2. Spring Data JPA — A Different DAO Approach

What JPA Actually Is

JPA (Jakarta Persistence API) is a specification for mapping Java objects to relational database tables. It is not the implementation itself. Hibernate is the commonly used implementation.

The problem JPA solves is repetitive object-to-row mapping. With hand-written JDBC, every DAO method may have to read columns from ResultSet and copy them into Java fields. JPA lets the mapping be declared once.

@Entity — Declaring the Mapping Once

Before JPA

// Earlier version — plain Java class

```
public class Employee {

    private int employeeId;
    private String name;
    private String department;
    private BigDecimal salary;

    // getters and setters
}
```

JPA Entity

```
import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity @Table(name =
"employees") public class
Employee {

    @Id @GeneratedValue(strategy =
GenerationType.IDENTITY) private int employeeId;

    @Column(nullable = false)
private String name;

    @Column(nullable = false)
private String department;

    @Column(nullable = false)
private BigDecimal salary;

    public Employee() {}

    // getters and setters
}
```

Entity-to-table mapping

Employee class		employees table
employeeId	<----->	employee_id PRIMARY
KEY name	<----->	name VARCHAR
department	<----->	department VARCHAR
salary	<----->	salary DECIMAL

The mapping is defined once on the Entity. DAO methods no longer need to manually call `rs.getString("name")` or `rs.getBigDecimal("salary")` for every operation.

Important distinction: an Entity is database-aware and managed by JPA. A DTO is a plain data carrier with no database awareness. Small applications sometimes use one class for both roles, while larger applications often keep them separate.

Interview Question

Q: What's the practical difference between a JPA @Entity and a plain DTO?

A: An Entity is managed by JPA and mapped to a database table. A DTO is a simple data carrier used to move information between layers and has no database awareness.

JpaRepository — Where the DAO Layer Nearly Disappears

Repository

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {

    // No implementation class is required.
}
```

JpaRepository provides common CRUD operations immediately.

Method	What It Does
save(Employee e)	INSERT or UPDATE
findById(Integer id)	SELECT by primary key
findAll()	SELECT all rows
deleteById(Integer id)	DELETE by primary key
count()	COUNT rows

Using the repository

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository
employeeRepository) {        this.employeeRepository =
employeeRepository;    }

    public List<Employee> findAll() {
return employeeRepository.findAll();    }

    public Employee save(Employee employee) {
return employeeRepository.save(employee);    }
}
```

What happens behind the interface

```
EmployeeRepository interface
|           v Spring Data JPA
detects it at startup
|           v Creates a runtime implementation
v Creates a runtime implementation
|           v Hibernate performs
persistence work
|           v
v Hibernate generates SQL
|           v
JDBC executes the SQL
```

Interview Question

Q: If EmployeeRepository is only an interface, what object actually runs when findAll() is called?

A: Spring Data JPA creates a runtime implementation for the repository interface. That implementation delegates to Hibernate, which generates SQL and executes it through JDBC.

Derived Query Methods

No implementation required

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {

    List<Employee> findByDepartment(String department);

    List<Employee> findByNameContaining(String namePart);

    List<Employee> findBySalaryGreaterThan(
        BigDecimal minSalary
    );

    List<Employee> findByDepartmentAndSalaryGreaterThan(
        String department,
        BigDecimal minSalary
    );
}
```

Method-name parsing

```
findByDepartmentAndSalaryGreaterThan(...)
)
    |
    |-- findBy
    |-- Department
    |-- And
    |-- Salary
    |--
    GreaterThan
v WHERE department = ?
AND salary > ?
```

Spring Data JPA reads the method name, recognizes keywords such as `findBy`, `And`, `Containing`, and `GreaterThan`, and matches property names against the Entity.

Best Practice: derived query methods work well for simple queries. When the method name becomes difficult to read because of many conditions or complex joins, use an explicit `@Query`.

✓ Good

`findByDepartmentAndSalaryGreaterThan(...)` for a simple two-condition query.

✗ Bad

A long method name containing many conditions that becomes harder to understand than the query itself.

Interview Question

Q: How does Spring Data JPA know what query to generate for `findByDepartmentAndSalaryGreaterThan`?

A: It parses the method name using recognized keywords and Entity property names, then creates the corresponding query and executes it through the generated repository implementation.

3. Spring JDBC — The Middle Ground

JdbcTemplate — Real SQL Without the Boilerplate

`JdbcTemplate` still lets you write SQL yourself, but it removes repetitive JDBC infrastructure such as connection handling, statement setup, resource cleanup, and checked SQL exception handling at every call site.

Hand-written JDBC //

```
Hand-written JDBC

public Employee getEmployeeById(int id) {

    String sql = "SELECT * FROM employees
WHERE employee_id = ?";

    try (Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```

        pstmt.setInt(1, id);

ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
Employee e = new Employee();

            e.setEmployeeId(rs.getInt("employee_id"));
            e.setName(rs.getString("name"));
            e.setDepartment(rs.getString("department"));
            e.setSalary(rs.getBigDecimal("salary"));

            return e;
        }

        } catch (SQLException ex) {
            System.err.println(
                "Query failed: " + ex.getMessage()
            );
        }

        return null;
    }

```

JdbcTemplate

```

@Repository
public class EmployeeJdbcDAO {

    private final JdbcTemplate jdbcTemplate;

    public EmployeeJdbcDAO(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public Employee getEmployeeById(int id) {

        String sql = "SELECT * FROM employees\nWHERE employee_id = ?";

        List<Employee> results =
            jdbcTemplate.query(sql, this::mapRow, id);

        return
            results.isEmpty()
            ? null
            :
            results.get(0);
    }

    private Employee mapRow(
        ResultSet rs,
        int rowNum
    )

    throws SQLException {

        Employee e = new Employee();

        e.setEmployeeId(rs.getInt("employee_id"));
        e.setName(rs.getString("name"));
        e.setDepartment(rs.getString("department"));
        e.setSalary(rs.getBigDecimal("salary"));

        return e;
    }
}

```

What JdbcTemplate Eliminates

- Manual Connection acquisition and closing.
- Repeated try/catch boilerplate around every DAO method.
- Manual PreparedStatement parameter setup.
- Forced checked SQL exception handling at every call site.

What JdbcTemplate Does Not Eliminate

- You still write the SQL.
- You still control the ResultSet-to-object mapping.
- A RowMapper is still required when mapping rows into Java objects.

RowMapper

Reusable RowMapper

```
public class EmployeeRowMapper
implements RowMapper<Employee> {

    @Override
    public Employee mapRow(
ResultSet rs,          int
rowNum      ) throws SQLException {
Employee e = new Employee();

        e.setEmployeeId(rs.getInt("employee_id"));
        e.setName(rs.getString("name"));
        e.setDepartment(rs.getString("department"));
        e.setSalary(rs.getBigDecimal("salary"));

        return e;
    }
}

List<Employee> all =
jdbcTemplate.query(
"SELECT * FROM employees",
new EmployeeRowMapper() );
```

Interview Question

Q: What does JdbcTemplate eliminate compared with hand-written JDBC, and what does it deliberately not eliminate?

A: It eliminates much of the connection, statement, resource-management, and checked-exception boilerplate. It does not eliminate the SQL or the ResultSet-to-object mapping.

4. Choosing Between the Three Approaches

Side-by-Side Comparison

Aspect	Hand-Written JDBC	JdbcTemplate	Spring Data JPA
SQL	You write it	You write it	Usually generated
Row mapping	You	You / RowMapper	Hibernate
Boilerplate	Highest	Medium	Lowest
SQL control	Full	Full	Reduced by default
Best fit	Learning / very custom SQL	SQL control with less ceremony	CRUD-heavy applications

There is no universally best option. Spring Data JPA is productive for standard CRUD-heavy applications. JdbcTemplate or hand-written JDBC can be the better choice when precise SQL control is important, especially for complex reporting queries.

Interview Question

Q: For a reporting feature requiring multiple joins and conditional aggregation, would you use a derived JPA query or JdbcTemplate?

A: JdbcTemplate, or an explicit JPA @Query, is generally a better choice. Derived method names become difficult to read as query complexity grows, while JdbcTemplate gives direct control over the SQL.

5. Practice: Rebuild Employee Payroll Using Spring Data JPA

1. Add the JPA Dependency

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

This starter brings Spring Data JPA, Hibernate as the JPA implementation, and transaction support into the project.

2. EmployeeRepository

```
package com.payroll.repository;

import com.payroll.dto.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.math.BigDecimal;
import java.util.List;

@Repository
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {

    List<Employee> findByDepartment(String department);

    List<Employee> findBySalaryGreaterThan(
        BigDecimal minSalary
    );
}
```

3. EmployeeService

```
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(
        EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    public List<Employee> findAll() {
        return employeeRepository.findAll();
    }

    public Employee findById(int id) {
        Optional<Employee> result =
            employeeRepository.findById(id);

        return
            result.orElse(null);
    }

    public Employee save(Employee employee) {

        // Business rule belongs in Service.
        if (employee.getSalary()
            .compareTo(BigDecimal.ZERO) <= 0) {

            throw new IllegalArgumentException(
                "Salary must be positive"
            );
        }

        return
            employeeRepository.save(employee);
    }
}
```

```
public List<Employee> findByDepartment(  
String department) {  
  
    return employeeRepository  
.findByDepartment(department);  
}  
}
```

4. application.properties

```
spring.jpa.hibernate.ddl-auto=update spring.jpa.show-  
sql=true
```

ddl-auto=update allows Hibernate to create or update the development schema from the Entity mapping. It is convenient during development, but production systems normally use controlled migration scripts instead of allowing Hibernate to change a real schema automatically.

Sample console output

```
Hibernate:      insert into  
employees      (department,  
name, salary)   values  
(?, ?, ?)
```

Important Note

EmployeeController can remain unchanged because it depends on the Service contract, not on the underlying DAO implementation. Replacing EmployeeDAOImpl with EmployeeRepository therefore does not require Controller changes.

Day 11 — Interview Questions with Detailed Explanations

Interview Question

Q: Why shouldn't business validation logic live in the Controller layer?

A: The Controller handles HTTP concerns. Business rules must also apply when the same operation is triggered by non-HTTP callers, so they belong in the Service layer.

Interview Question

Q: What is JPA, and is it a library you add to the project?

A: JPA is a specification for persistence and object-relational mapping. Hibernate is a common implementation. The Spring Boot JPA starter brings the required JPA and Hibernate components together. **Interview Question**

Q: What's the practical difference between a JPA @Entity and a plain DTO?

A: An Entity is managed by JPA and mapped to a database table. A DTO is a plain data carrier used to move data between layers without database awareness.

Interview Question

Q: If EmployeeRepository is only an interface, what actually runs when findAll() is called?

A: Spring Data JPA provides a runtime implementation for the repository interface. That implementation delegates to Hibernate, which generates SQL and executes it through JDBC.

Interview Question

Q: How does Spring Data JPA know what SQL to generate for a derived query method?

A: It parses the method name using recognized keywords and matches the property names against the Entity fields, then builds the corresponding query.

Interview Question

Q: What does JdbcTemplate eliminate compared with hand-written JDBC?

A: It removes much of the connection, statement, resource-management, and checked-exception boilerplate while leaving SQL and row mapping under developer control.

Interview Question

Q: For a complex reporting query with multiple joins and conditional aggregation, which approach is better?

A: JdbcTemplate or an explicit JPA @Query is generally more suitable because complex SQL is difficult to express clearly through derived method names.

Interview Question

Q: Why was hand-written JDBC taught before Spring Data JPA?

A: It makes the underlying mechanics visible. Once JDBC is understood, JpaRepository is easier to understand because the developer knows what persistence work is being automated.

Interview Question

Q: What does spring.jpa.hibernate.ddl-auto=update do?

A: It allows Hibernate to create or update the database schema based on the Entity mappings. It is convenient for development but not normally appropriate as an uncontrolled production schema-management strategy.

Interview Question

Q: Why did EmployeeController require no code changes when the data layer moved to Spring Data JPA?

A: The Controller depends on the Service interface and behavior, not on the DAO implementation. The Service changed internally while keeping the same methods used by the Controller.

Day 11 — Problems to Solve

Problem 1 — Convert Contacts App to Spring Data JPA

- Convert Contact into a proper @Entity with @Id, @GeneratedValue, and @Column where needed.
- Replace ContactDAO and ContactDAOImpl with ContactRepository extends JpaRepository<Contact, Integer>.
- Recreate findByNameContaining(String name) as a derived query method with zero implementation code.
- Confirm that ContactController and ContactService method signatures do not need to change.

Problem 2 — Prove the SQL Is Real

- Enable spring.jpa.show-sql=true.
- Call POST /contacts, GET /contacts/{id}, and DELETE /contacts/{id}.
- Copy the SQL Hibernate prints for each call.
- For findById(), confirm that the generated SELECT contains the expected WHERE condition.

Problem 3 — Break a Derived Query Method on Purpose

- Add findByDeptment(String department) with a deliberate spelling error.
- Start the application and read the startup error.
- Fix the typo and confirm that the application starts.
- Compare this failure with a typo inside a raw SQL string from the earlier JDBC implementation.

Problem 4 — Add a Complex Query with @Query

- Add a repository method for department and salary range using JPQL.
- Expose GET /employees/search?department=Engineering&min=50000&max=100000.
- Test employees both inside and outside the range.
- Compare the readability of the @Query version with the equivalent derived method name.

Problem 5 — Rebuild One DAO Using JdbcTemplate

- Choose one entity and implement getAll and getById using JdbcTemplate.
- Use a RowMapper for ResultSet-to-object mapping.
- Compare the result with the earlier hand-written DAO.
- Write a short comparison of hand-written JDBC, JdbcTemplate, and JPA.

Problem 6 — Deliberately Misplace Logic, Then Fix It

- Move the salary-positive check from EmployeeService into EmployeeController.
- Create a second entry point that calls EmployeeService.save() directly, simulating a batch process.
- Call the batch path with a negative salary and observe that Controller validation is bypassed.
- Move the rule back into the Service and verify that both HTTP and batch callers enforce it.

Day 11 — Key Points

- Controller handles HTTP concerns; Service handles business rules; DAO/Repository handles persistence.
- JPA is a specification; Hibernate is a common implementation.
- @Entity defines the object-to-table mapping.
- JpaRepository provides CRUD operations without a hand-written implementation class.
- Derived query methods are useful for simple queries and are parsed from method names.

- JdbcTemplate keeps SQL control while removing much of the JDBC boilerplate.
- There is no universally best data-access approach; the correct choice depends on the requirement.
- Replacing a DAO implementation should not force Controller changes when the layers are properly separated.

Conclusion

Day 11 connects the architecture learned in the earlier days with practical Spring data-access choices. The important skill is not simply knowing JPA or JdbcTemplate, but understanding what responsibility belongs in each layer and choosing the data-access approach that fits the problem.